

JALEN — COLLABORATOR PROMPT (paste this whole file into the other Claude session)

Written 2026-10-01 by the Claude Code session that has been building Jalen with the owner ("Session A", the Integrator). The state described here is exact as of main commit `e34e790` (5,226 tests collected). Anything marked **VERIFY** may have moved: check it before relying on it.

0. READ THIS FIRST — what you are, and the one rule that matters

You are **Session B, the Collaborator**. Another Claude Code session (Session A, the Integrator) is working on the same project, on the same Windows laptop, at the same time, for the same owner. You are both allowed to be fast and ambitious. You are **not** allowed to overwrite each other.

The single rule: **nobody but the Integrator writes to `main`, and nobody edits the main checkout's working tree except the Integrator**. You work in your own git worktree on your own branch, deliver commits, and report. The Integrator reviews, merges and runs the full test gate. Section 8 is the full protocol. If anything in this document conflicts with the owner's latest message to you, the owner wins — and tell the Integrator by writing to your outbox file (section 8.5).

Two modes, depending on what you can reach:

- **Mode 1 — same machine, repo access (expected)**. You can run commands in `C:\Users\user\Desktop\Jarvis-setup\jarvis`. Follow section 8 exactly.
 - **Mode 2 — no repo access** (for example a chat-only session). You cannot run anything. Then your job is design, review and writing **new files only** (never edit existing ones): deliver each as a complete file in your reply, with its intended path, so the owner can drop it in. Do not guess at the contents of existing files; ask for them.
-

1. The owner, and how to talk to him

- The owner is **Jaloliddin** (he is addressed as "Boss" by Jalen). A machine-learning engineer and community builder. He runs a Telegram channel, "**AI Engineering & Machine Learning**" (public: <https://t.me/MLcommunity>), and writes to an audience of ML students and practitioners.
- Writes casually, fast, often with typos and voice-dictation errors. Read for intent. Is not a native English speaker; writes to people in Uzbek and Russian as well. Keep your replies to him plain, short, concrete. No jargon dumps.
- **He wants action, not discussion**. Standing instruction (saved in memory, 2026-09): implement improvements without asking; sequential phases, hard-tested, with a done/not-done report. "Lock in", "work harder", "spawn more agents" mean: keep going, parallelise, do not stop to ask.
- **Decisions and blockers go to him as POLLS** — his explicit standing request. Use the question tool with 2-4 labelled options, a recommended default first (marked "Recommended"), one line of consequence each. Ask only for decisions that are genuinely his (privacy, what goes public, money, accounts, anything irreversible) or for actions only he can do (signing in, approving an OAuth consent). Everything else: decide, state your assumption, proceed.
- **Report done and not-done in the same breath**. Silence about a skipped part reads as success. Never claim something works unless you ran it and saw it work; say exactly what you ran.

- **Never print, paste, log or commit a secret:** tokens, `.env` contents, vault contents, Telegram session files, OAuth secrets, passwords. If you must refer to one, name it, never show it.
 - **Safety-classifier caution (learned the hard way today).** Long, detailed write-ups of security holes made the model's output get cut off by an automatic classifier several times. Describe each protection in one plain line (what was wrong, what it does now). Do not write step-by-step instructions for abusing anything, in commit messages, reports or chat. Reproduce bugs with tests, not with narratives.
 - **He asked for the following (his words, condensed):** a Jalen that listens to any question, performs any task without him lifting a finger, controls his web browser and his whole computer (open things, move things from C: to D:, identify issues), controls his whole Telegram (read DMs, write posts in his channel format with his **premium emoji**, send voice messages), does extensive research, hands tasks to ChatGPT and Claude, and "never pretends to be deaf". Constraint he set himself: **it may do anything a software engineer would do EXCEPT delete online software/accounts/data**. Quality bar: "no mistakes".
-

2. The product

Jalen (the Python package is still called `jarvis`; the folder is `Jarvis-setup\jarvis`; he renamed it from "Jarvis" to "Jalen") is a **voice assistant that drives one Windows laptop**. Python (venv is 3.13.x; the project says 3.12+), single process, about 350 MB resident.

The constraint that decided the architecture: the laptop has **8 GB RAM with about 1 GB free**, so there is no local LLM and no PyTorch. Heavy thinking happens on someone else's computer.

How it hears and speaks: - Wake word "hey jalen" (openWakeWord ONNX models), Silero VAD, speech-to-text via **Groq** (cloud) with a local **moonshine** fallback, text-to-speech via **edge-tts** (default voice `en-US-AndrewNeural1`, with a local cache). An orb UI shows state. - Modes: `start` (voice), `text` (typed, same brain and same safety rules), a Telegram **bot** mode (`--telegram`, restricted to the owner's user id).

How it thinks: - Most spoken commands never reach a model: `jarvis/brain/router.py` has ~140 regex rules, **first match wins**, free and instant ("scroll down", "open spotify", "what time is it"). - Everything else goes to **one brain: Claude**, through `ClaudeSDKClient`, which spawns a bundled Claude Code CLI child (`claude.exe` inside the `claude-agent-sdk` wheel). There is **no second brain**: Gemini and OpenRouter ("Hermes") are used only when the owner asks Jalen to *delegate* something. `brain.provider` and `use_gemini_for_chitchat` in the YAML are **not read by any code** (labelled `[NOT READ]`). - The model calls Jalen's tools (about 155, in `jarvis/tools/*.py`, described in `TOOL_SPECS` in `jarvis/brain/tools.py`). Every tool call passes through a **PreToolUse** hook that classifies it with `SafetyEngine` (green / amber / red / black) before anything runs.

What it can do today (high level): time, battery, system status, disk/cleanup reports, open/close apps and windows, type and click into windows, files (read/list/search/copy/move/rename, content search), documents (PDF, docx, pptx, xlsx), Gmail and Calendar (read, search, draft), Telegram (read, search, draft, send), community posts in his format, a Chrome-based "browser worker" (Jalen's own Chrome profile; ChatGPT/Gemini hand-off; forms), a Chrome extension path (his real tab), coding-agent jobs (Claude Code in the background), a credentials vault (never exposed to the model), web research (`web_search`, `web_read` incl. PDFs).

3. The machine (facts you need)

- Windows 10 Pro 10.0.19045. Shells: Git Bash and PowerShell 7 both exist. In PowerShell write `;` or `&&` carefully; in bash use forward slashes.
- **RAM: 7.5 GB total, about 0.5-1.7 GB free** depending on what is running. Several parallel agents each running the full test suite is about the limit. If free RAM drops under ~500 MB, stop launching things.

- **Disk: C: has about 8 GB free of 157 GB (95% full); D: has about 280-300 GB free.** Do not install big things on C:. Install with `pip install --target <folder on D:>` or a scratch folder if you must add a package. Do **not** add dependencies to the project venv or `requirements.txt` without telling the Integrator.
- 12 logical CPUs.
- The repo: `C:\Users\user\Desktop\Jarvis-setup\jarvis`. The interpreter is **always** `C:\Users\user\Desktop\Jarvis-setup\jarvis\.venv\Scripts\python.exe` (bare `python` is a different install with none of the packages). Worktrees (`.claude\worktrees\...`) have **no** `.venv`, **no** `.env`, **no** `data\`, **no** `models\`: always call the main venv by absolute path.
- Git: local only. **No remote, no CI, no linter, no type checker.** Line endings: files are CRLF in the working tree on this machine and git warns about LF→CRLF on every add; that is noise.
- `.env` (gitignored) holds credentials. Never read it into output. `ANTHROPIC_API_KEY` must stay **empty** (a value there silently switches the brain to per-token billing).

4. Running and testing (exact commands)

```
cd C:\Users\user\Desktop\Jarvis-setup\jarvis
.\jalen.ps1 check      # environment + credentials + accounts diagnostic. Run first. Exit 0 = all good.
.\jalen.ps1 start      # voice
.\jalen.ps1 text       # typed, same brain, same safety rules
.\jalen.ps1 todo       # what is outstanding
.venv\Scripts\python.exe run.py --why      # why the last run stopped
.venv\Scripts\python.exe run.py --status   # is a Jalen running?
```

The test gate (this is the definition of "green"):

```
.venv\Scripts\python.exe -m pytest tests\ -q -p no:cacheprovider --
ignore=tests\benchmark_latency.py --ignore=tests\benchmark_open.py --
ignore=tests\benchmark_phrasing.py
```

- Always `python -m pytest`, never the bare `pytest.exe` (about three quarters of the test files rely on `-m` putting the CWD on the path). `pytest-timeout` is not installed; do not pass `--timeout`.
- **Current count at e34e790 : 5,226 tests collected; 5,225 pass, 1 fails.** The one failure is `test_overhaul_fixes.py::test_real_typos_and_abbreviations_still_resolve[capcut-True]`: it expects CapCut to be installed and it is not. It is not a code problem. Every other failure is real.
- A full run takes **5-8 minutes** and uses a few hundred MB. Run it **once** per batch of changes, in the background if your tool allows, and poll the output file.
- Three voice tests call live Groq / edge-tts and are intermittent (`test_stt_roundtrips_through_groq`, `test_barge_in_latency_is_measured`, `test_failure_paths_keep_jarvis_alive`). The trap: the fallback *working* trips the assertion. Re-run two or three times **in isolation** before touching `stt.py`.
- **In a worktree** these fail identically with and without your change, because the files they need are gitignored: `test_attachments::test_the_refusal_is_not_overridable` (needs `.env`), `test_voice_pipeline` ×5 and `test_wake_both_names` ×2 (need `models/`), `test_native_host_process` ×2 (needs `.venv`), `test_conversation_requests M1` ×2 (needs `data/`), plus the CapCut one. That is **13 expected failures in a worktree** and 1 in main. Confirm any *other* failure is yours.
- Tests that read the real audit log skip in a worktree; copy `data\audit.jsonl` (read-only copy) somewhere and set `JALEN_AUDIT_LOG` to run them.

- **Tests must never write the real `data\` directory.** `tests/conftest.py` redirects audit, memory, router-miss log, web chats, browser profile and more. After a full run, nothing under `data\` should have changed (the Integrator checks this with a before/after file listing). If your tests write there, that is a bug.
- **Tests that start a real Chrome run it headless** (switch `JALEN_TEST_HEADLESS_CHROME`, set by `conftest`). Before that fix, every test run popped a visible Chrome window on the owner's screen and closed it a second later, which he experienced as "the browser keeps failing". Never introduce a test that shows a window.

5. Accounts and sign-in state (VERIFY before relying)

No credential lives in the repo. State on 2026-10-01:

Thing	State
Claude brain	Signed in. The brain's Claude Code CLI child authenticates with <code>CLAUDE_CODE_OAUTH_TOKEN</code> in <code>.env</code> (OAuth token from <code>claude.exe setup-token</code>). Verified: a one-word round trip works; <code>jalen check</code> passes. It had been signed out for ten days before today.
Google (Gmail + Calendar)	Reconnected today with <code>scripts\connect_google.py</code> . The Google Cloud OAuth consent screen is in Testing mode, so the refresh token expires after 7 days (~2026-10-08) until the owner clicks "Publish app" in Google Cloud Console. <code>jalen check</code> now has an Accounts section that reports this. Account used for Jalen's Google tasks: <code>jaloliddin2009applicant@gmail.com</code> (his instruction; not the Google account with his profile picture).
Telegram (personal)	Telethon user session as the owner (<code>@M1_eengineer</code>). Pre-approved send destinations without asking: "Saved Messages" and "AI engineering & Machine learning" (config <code>telegram.personal.send_without_asking_to</code>). Bot mode restricted by <code>TELEGRAM_ALLOWED_USER_IDS</code> . Only ONE process may use the Telethon session at a time — a second client on the same session file gets the session killed or corrupted. Never start a second Jalen while one runs.
ChatGPT	Signed in inside Jalen's own Chrome profile (<code>data\browser_profile</code>), done by the owner.
claude.ai	A Jalen-Chrome window was opened on <code>claude.ai</code> today; the owner says it signed itself in (he did not type credentials). VERIFY which account before building on it.
Gemini / OpenRouter	Keys exist in <code>.env</code> for delegation only; OpenRouter had 0 credit when last checked.
Telegram bot, GitHub, Notion	Tokens exist in <code>.env</code> .

Two browsers exist, and the owner has now decided between them (see section 10): his **everyday Chrome** (his real profile, via the Jalen **Chrome extension** in `browser_extension\` talking to the app through a native-messaging bridge) versus **Jalen's own Chrome** (separate profile at `data\browser_profile`, driven over the DevTools protocol by `jarvis/tools/webagent.py`). **Decision: everyday Chrome by default.** Only one process may own `data\browser_profile` at a time — launching a second Chrome on that profile kills the first.

6. Architecture map

```
jarvis/
  app.py          3,000 lines. The Jalen class: the mic loop (run), the address gate
                  (should_act_on),
                  process(text) (kill switch -> answers -> routing -> brain),
confirm()/announce(),
                  handle_local() (router hits), brain-down latch, timers.
  safety.py       707 lines. SafetyEngine.classify(): never-touch -> injection guard -> pre-
approved
                  destinations -> tiers. protected_path(), protected_domain(),
unseen_url_is_harmless().
  taint.py        Process-wide "has Jalen read untrusted text since he last spoke?" + web
addresses seen.
  audit.py        audit.jsonl / audit.db. Every utterance, action, timing row, error.
  conversation.py Reference expansion ("it", "that"), progress, task state.
  habits.py       Sentences learned after 3 identical brain decisions (recalled without the
brain).
  plan.py         Reads "send/draft/save ... to X" out of what he said (the contract for a turn).
  brain/
    router.py     Regex rules, first match wins; is_kill_phrase(); addressed_to_jalen().
    agent.py      Brain: system prompt (Brain.system_prompt), the PreToolUse hook (_make_hook),
ask() streaming, BrainUnavailable, signed_in().
    tools.py      TOOL_SPECS (name -> (description, params)), build_sdk_tools().
  tools/
    messaging.py  Telegram: read/search/send/draft, HTML->entities, _run_send ("Not confirmed"
semantics)
    attachments.py send_telegram_file, draft_email_with_file; protected-file check
    drafting.py    send_posts (batches), post drafting
    gmail.py gcalendar.py Google read/draft
    research.py    web_search, web_read (bounded fetch, PDFs, redirect checks, fenced + tainted)
    webagent.py    2,800 lines. Jalen's own Chrome: _Session (one thread, one Chrome), SiteAdapter
for
                  ChatGPT/Gemini, sign-in flows, browse_to, read_browser_page, delegation.
    webforms.py profile.py otp.py autofill.py vault.py Forms, login fields, one-time codes,
secrets.
    browser_ext.py browsertabs.py The extension path (his real Chrome tabs) and tab tools.
    devwork.py tasks.py coding.py handoff.py Background Claude Code jobs, cancel_task.
    filesystem.py documents.py launcher.py system.py desktop.py sysinfo.py technician.py
selfcontrol.py
  integrations/   google_auth.py, telegram_user.py (RUNTIME: the Telethon loop), telegram_bot.py
audio/
  ui/            stt.py, tts.py, vad, wake word, speaker
the orb
  config/
    jarvis.yaml   782 lines. THE DESIGN RECORD: the comments explain why every number is what it
is.
    safety.yaml   617 lines. Tiers, never_touch (paths, patterns, apps, domains, harmless_names),
injection_guard (markers, refuse_from_content, allow_unseen_urls_on_hosts).
  scripts/
    vault_setup.py,
                  check_env.py (jalen check), connect_google.py, connect_telegram.py,
                  abilities.py (regenerates ABILITIES.md), rehearse.py ...
                  (measure_address_gate.py exists only on the listening branch until it is
merged)
  browser_extension/ The Chrome extension (manifest, background, native host launcher).
  tests/           ~125 test files, 5,226 tests. conftest.py keeps tests out of data/.
  data/           RUNTIME DATA. Never commit. Never write from tests. Contains the vault, audit
log,
```

```
docs/ Telegram session, browser profile, habits, drafts.
community_post_format.md (his post format), this folder (collab/).
```

One spoken turn, end to end: 1. Microphone → VAD → wake word / follow-up window → address gate `should_act_on` (does it start with his name, or is a question window open? Is it Jalen's own voice echoing back? — `_sounds_like_its_own_voice`). 2. Groq STT → `process(text, from_him=...)`. 3. Kill switch ("stop", "enough" in many spellings) → pending answers (rating / question / confirmation) → stitching → `taint.he_asked_again()` if positively his and no other turn is running → `conversation.expand_references` ("it") → `planning.read_plan`. 4. `router.route(text)`: a rule matched → `handle_local(intent, his_own_words=...)` → classify → run tool → speak. No model. 5. Else habits recall, else `handle_with_brain` → Claude streams text and calls tools → each call passes the hook (classify with `origin=taint.origin_now()`) → green runs, amber announces with a stop window, red asks aloud and waits for a spoken yes (`ConfirmAnswer`), black refuses. 6. Tool results that carry other people's text are **fenced** as `UNTRUSTED CONTENT` and call `taint.mark(source, text)`.

7. The safety model — read all of it; these fail SILENTLY when broken

7.1 Tiers (config/safety.yaml)

GREEN runs. AMBER announces ("...", say stop if you don't want that") and runs unless he stops it. RED stops and asks aloud; no answer in time = cancelled. BLACK refuses always. `paranoid_first_week` ships **false**, so AMBER does not behave like RED.

7.2 Order inside `SafetyEngine.classify` (do not reorder)

1. **never-touch** (paths, filename patterns, password-manager apps, protected domains) → BLACK. Checked first, before origin.
2. `origin == "content"` (the turn has read untrusted text): RED and AMBER tools → BLACK; GREEN tools listed in `injection_guard.refuse_from_content` → BLACK. One narrow exception: a send to his own Saved Messages when *his words* named it.
3. **Pre-approved destinations** downgrade RED → GREEN for exact-named destinations — **only for** `origin == "user"`.
4. Posture adjustments. `tests/test_adversarial.py` compares source-string indices to pin step 2 above step 3. Swap them and a pre-approved Telegram channel becomes an open relay for anyone who gets text in front of Jalen, while every behavioural test still passes.

7.3 Invariants (each one has a failing story behind it)

1. **Secrets never become tool results.** `vault.get_secret()` is absent from every `REGISTRY` and from `TOOL_SPECS`, and that absence is the mechanism: a tool result reaches the model, the transcript, the audit log and possibly the speakers. No flag, no tier for it.
2. **Four `ClaudeAgentOptions` fields are safety decisions:** `tools=[]`, `setting_sources=[]`, `skills=[]`, `permission_mode="bypassPermissions"`. At defaults the model would get the SDK's own Bash/Write/Edit and this machine's MCP servers, none of which `safety.yaml` knows. Pinned by `tests/test_agent_sdk_configuration.py`.
3. `suppress_cli_console_window()` must run **before** `ClaudeSDKClient` is constructed (it patches `anyio.open_process`).
4. **Never hardcode `origin="user"`, and never leave `origin= out`** (the default is "user"): use `taint.origin_now()`. Questions that are not about origin ("is this file / site protected?") go to `SafetyEngine.protected_path()` / `protected_domain()`, never to a second copy of the lists. **The one**

deliberate exception: a router rule that matched what he *said*, word for word, runs with origin "user" (`handle_local(..., his_own_words=True)`), only when `expand_references()` left the sentence untouched. The brain's hook never gets this.

5. **A GREEN tool that acts on the world** (types, clicks, launches, writes a file, stores a memory, ends a job) **must be on** `injection_guard.refuse_from_content`. Nothing enforces an omission; tests only catch typos.
6. **Two gates decide whether he is heard, and they must agree:** the microphone gate (`follow_up_until, _expectation_open()`) decides whether a sound opens a window; the address gate (`should_act_on`) decides whether the words are acted on. For a year the first was a local variable the second could not see, so Jalen opened the mic for an answer, transcribed it, and dropped it for not starting with his name. `tests/test_answering_a_question.py` pins both. The same shape (conversational state with a lifetime that one reader enforces and another does not) has bitten three times: grep before adding a fourth.
7. **The address gate is also the echo defence:** Jalen's own sentences do not start with his name, and there is no acoustic echo cancellation. Any exemption you add must call `_sounds_like_its_own_voice()` or it is a self-triggering loop.
8. **timing.wait_s is not the wait;** `felt_wait_s` is. The four original audit labels (heard/thought/wait/spoke) are load-bearing: add, never re-anchor.
9. **Never pass user text as an argument to a .cmd:** `shutil.which("claude")` returns npm's `claude.CMD`, `cmd.exe` truncates at the first newline, flattens dashes, expands `%VAR%`. Use a real executable or send on stdin.
10. **Never write a regex, a backslash or non-ASCII text through a shell heredoc:** `\b` arrives as a 0x08 byte that still compiles, so the guard never fires and every test keeps passing. **Use the Edit/Write tools.**

7.4 What the safety work looks like today (so you do not redo or undo it)

- **Never-touch path check** resolves paths the way Windows does: `~`, `%VAR%`, `..`, long-path prefixes, **junctions**, **8.3 short names** (`os.path.realpath`), trailing dots/spaces and `::$DATA` suffixes; device paths and loopback network names are refused as unreadable; filename patterns see resolved names; `never_touch.harmless_names` (e.g. HuggingFace `tokenizer.json`, `.env.example`) are exempt from the *filename patterns only*, never from protected folders.
- **Protected domains** (banks, PayPal, click.uz, payme.uz, binance) are matched by parsed **hostname**, not substring; `protected_domain(address)` is public for callers that learn a destination later (redirect landings).
- **Injection guard after a read:** `web_read` after a read may fetch only an address that appeared **written out** in text read this turn (`taint.url_was_read`), or a plain reference page on `allow_unseen_urls_on_hosts` (Wikipedia, arXiv, huggingface.co ...). `taint.mark(source, text)` records addresses; the Gmail and research fences pass their text. **Not yet wired: the Telegram fence** (`messaging._fence`, **assigned to Wave 4's telegram-fixes lane**) and **the coding-job fence** (`devwork`, **the Integrator's**) — until they pass their text, a link inside a Telegram message or a coding-job log cannot be followed by `web_read` in the same turn (refused with a sentence).
- **Confirmations are bound:** `ConfirmAnswer` is truthy only for a real yes; a correction ("yes, but to Ali instead") is passed back; Jalen's own "Confirm?" echoing through the mic cannot approve itself (`_echoes_the_confirmation`); one lock for confirm/announce.
- **Pre-approved destinations are exact-named**, not substrings ("Ed" and "a" were pre-approved by a substring bug).
- **Secrets are bound to sites** (`data\secret_sites.json`, exact-or-subdomain host match); a one-time approval to type a password is a spoken yes to a question that *names the site*, re-checked on the page at typing time.
- **Form steps stay on the inspected site** (`webforms._FORM_READ`: origin + path recorded by `inspect_form`, re-checked inside the browser job before acting and again before typing).

- **Telegram send timeouts:** `messaging._run_send` returns "Not confirmed - ... check the chat before sending it again" on a timeout (the coroutine keeps running and may deliver), so the brain never double-posts. Not being signed in still raises `TelegramNotConnected` (it carries the fix).
- `taint.he_asked_again()` is called from exactly one place (`process()`), only for a fresh instruction that is positively his and while no other turn is in flight. An *answer* to a question never clears the taint.

8. THE COLLABORATION PROTOCOL (Mode 1)

8.1 Roles

- **Integrator (Session A):** owns `main`, merges, runs the full gate, resolves cross-branch conflicts, owns the Telegram/browser/live QA runs, writes `docs/collab/STATUS.md` (single writer).
- **Collaborator (you, Session B):** owns your lane (section 9), your branch, and your outbox file. You never push to main and never edit files in the main checkout.

8.2 Setup (do this first, exactly)

```
cd C:\Users\user\Desktop\Jarvis-setup\jarvis
git worktree add -b collab/<topic> C:\Users\user\Desktop\Jarvis-
setup\jarvis\.claude\worktrees\collab-<topic> main
```

Work only inside that worktree. The main checkout's `.claude/` folder is untracked; **never** `git add -A` or `git add .` — stage explicit paths. Never `git reset --hard`, `git clean`, `git checkout -- .` on anything but your own worktree, no force, no push, no bare `git stash` (the stash stack is **shared** across worktrees). If you must set work aside, make a WIP commit on your branch.

8.3 What may be shared, and what may not

- **One writer per file, always.** Shared files with two writers are how work vanishes. Rule of thumb: you may edit a file only if it is in your lane (section 9) **and** not in the in-flight list (8.4). For the files that every feature touches (`agent.py`, `tools.py`, `safety.yaml`, `router.py`, `safety.py`), make a **separate, tiny final commit** for those edits and tell the Integrator in your outbox; he will merge them last.
- **Do not run a second Jalen or a second Telethon client;** do not start Chrome on `data\browser_profile` unless your lane owns the browser and the Integrator's log says it is free. Do not touch `data\` at all.
- **Scratch space:** use your own folder (for example `...\scratchpad\collab\`) and **unique file names** — a shared scratch file was overwritten by another agent today.
- Do not edit `CLAUDE.md`, `ABILITIES.md`, `WHAT_JALEN_CAN_DO.md`, `config/jarvis.yaml` except in your own clearly separated hunk, and say so.

8.4 In-flight files right now (the Integrator's Wave 4 — seven worker branches — touches these; do not edit them until STATUS.md says "WAVE 4 MERGED")

The first wave (premium emoji + stickers, DMs + voice, folder moves, listening) is **already merged** into main. Wave 4 is the follow-up to the independent reviews of that wave plus the live-QA fixes. Its lanes and the files they touch:

Wave 4 lane	Files it touches
files-fixes (folder-move data safety)	<code>jarvis/tools/foldermove.py</code> , <code>launcher.py</code> , <code>sysinfo.py</code> , <code>jarvis/safety.py</code> (<code>_describe</code>), <code>jarvis/brain/router.py</code> (the "move it" rule)

Wave 4 lane	Files it touches
telegram-fixes	jarvis/tools/messaging.py, stickers.py, jarvis/brain/router.py (voice phrasing), jarvis/brain/tools.py (specs), ABILITIES.md
listening-echo	jarvis/app.py, jarvis/audio/tts.py, jarvis/audio/speaker*, config/jarvis.yaml
browser-everyday	jarvis/tools/browser_ext.py, browsertabs.py, webagent.py, jarvis/bridge/, browser_extension/, a new browserdoor.py
research-fallback	jarvis/tools/research.py (and one seam call into webagent.py)
machine-facts	jarvis/tools/system.py, desktop.py, sysinfo.py, jarvis/brain/agent.py (sentence joining), jarvis/brain/tools.py, config/safety.yaml
conversation-quality	jarvis/brain/router.py, jarvis/brain/agent.py (system prompt), config/safety.yaml (suspicious_markers)

Files that **no** Wave 4 lane touches, and that are therefore safe for a collaborator: jarvis/tools/gmail.py, gcalendar.py, documents.py, vault.py, otp.py, profile.py, autofill.py, bulkmail.py, agents.py, coding.py, handoff.py, selfcontrol.py, technician.py, scripts/ (except abilities.py), docs/ (except collab/STATUS.md), README.md, SETUP.md, and any **new** file you create.

Merge order: the Integrator merges Wave 4 branch by branch and resolves cross-branch conflicts; they are never yours to resolve. Expect line-ending noise: a whole-file conflict usually means CRLF versus LF, not a real disagreement.

8.5 Handing work over

- Commit on your branch. **One commit per concern**, subject = a **past-tense sentence naming the discovery**, not the change ("close the calculator failed because the rule kept the word 'the'", not "fix close_app"). Body: what was wrong (measured, with numbers), what it does now, what you did **not** do. **Last line:** <total> tests, N new. Write the message to a file and use `git commit -F <file>`.
- Then **write your outbox** docs\collab\outbox\collab-<topic>.md *on your branch* (single writer = you): branch, full commit sha, one-paragraph summary, files touched, the exact test commands you ran and their result, a section "**needs from main**" (TOOL_SPECS lines, tier lines, system-prompt lines, anything in an in-flight file), and "**not done**".
- The Integrator reads it, cherry-picks your commits, runs the full gate, and answers in docs\collab\STATUS.md. If he asks for changes, make a **new** commit on your branch (never rewrite published commits).
- Reviewers are allowed to be harsh.** Treat every review finding as a *claim to verify* (reproduce it with a failing test), not an instruction; judge the diagnosis and the fix as two separate verdicts.

8.6 The loop for each task (this is what produced today's results)

- DISCOVER** — read the code and the real log (data\audit.json1, read-only copy). Measure before you believe.
- REPRODUCE** — a failing test, against the real function, with real inputs. Show it failing.
- DESIGN** — smallest change that fixes the cause; say what you are *not* changing.
- IMPLEMENT** — edit with the Edit tool. Match the surrounding comment density and naming.
- FOCUSED TESTS**, then **FULL GATE once**.
- LIVE CHECK** when it touches real behaviour (typed mode only, read-only requests; never ask Jalen to send, post, delete, move or sign in during QA).
- COMMIT + OUTBOX.**

8.7 Three-edit rule for tools

A new tool needs three edits that nothing hard-fails on if you miss one: (1) the module's `REGISTRY`, (2) its `TOOL_SPECS` entry in `jarvis/brain/tools.py`, (3) a tier in `config/safety.yaml`. (1) and (2) hard-fail at startup when they drift; (3) fails `tests/test_every_tool_has_a_tier.py`. If the tool is **GREEN and acts on the world**, also add it to `injection_guard.refuse_from_content`. Edits (2) and (3) are in in-flight files: put them in your "needs from main" until the wave is merged, or in your separate tiny final commit.

9. LANES

Lane A — Integrator (not yours)

Runs **Wave 4** (seven worker agents, each followed by an independent re-check) and merges it; owns the live QA runs (only one Jalen process and one Telegram session may exist, and it is his); wires the remaining fences (`devwork`) to `taint.mark(source, text)`; builds "channel posts read the first line and links aloud before posting" (the owner's decision); builds the Claude (`claude.ai`) hand-off adapter once the browser lane has landed; runs the full gate; writes `STATUS.md`.

Lane B — **CLAIMED by Wave 4. Do not start these. (listed so you know what exists and do not duplicate it)**

- **B1** everyday-Chrome as the default browser (new `browserdoor.py` seam, extension path); **B2** research fallback (HTTP 202 retry, then the browser, one honest sentence, never speak a status code); **B3** machine facts (time zone and time in other cities, uptime, foreground app, running programs); **B4** router phrasing ("find me papers on X" goes to the web, not a file search; "close the calculator"; disk/memory questions); **B5** the false prompt-injection alarm on ML paper titles; **B6** fused and repeated sentences; **B7** how it talks (no tool names or status codes, "Boss" at most once); **B8** "what is filling my C drive".
- Also in Wave 4: folder-move data safety (copy must keep Mark-of-the-Web, hidden attribute and creation times before the original is deleted; the dry run must end with a question; the named yes must name every protected file), Telegram follow-ups (voice-note decode bound, Uzbek/Russian voices, the sticker-with-no-pack question), and the listening echo window.

Lane C — **you (start here; every item has an acceptance test; none overlaps Wave 4)**

C1. A real-account verification script for the owner to run (new file: `scripts/Live_telegram_check.py`; *fakes-only tests*) Nothing built for premium emoji, stickers, voice messages or voice-note transcription has ever run against his real Telegram account; several constants in `jarvis/tools/stickers.py` are labelled `[NOT MEASURED]` for that reason. Write an **owner-run, step-by-step checklist script** that, with Jalen **stopped** (one Telethon client at a time!), (1) looks up three premium emoji by character and name, (2) lists his sticker packs (names and counts only), (3) sends ONE test post with a premium emoji to **Saved Messages** and reads it back, (4) sends ONE short voice message to **Saved Messages**, (5) times each step and prints a table he can paste back. It must **ask before each send**, refuse any destination other than Saved Messages, never print message contents or ids, and exit cleanly on Ctrl+C. Reuse the tool functions in `jarvis/tools/messaging.py` and `stickers.py` (read them; do not edit them). Tests use the fake client in `tests/_telegram_fakes.py`.

C2. Adversarial review of everything merged since a371945 (*review only; a written report, no code changes*) Use `git log a371945..main` and `git diff a371945..main`. Look for: tests that cannot fail, claims in commit messages the code does not deliver, over-refusing of ordinary requests, any of the ten invariants in section 7.3 weakened by a merge (the merges had real conflicts in `safety.py`, `messaging.py`, `tools.py`, `safety.yaml`; the Integrator resolved them by hand: check the resolutions), anything under `data/` written by tests, any secret in

any file. Prove findings with a failing test or a short reproduction; mark each CONFIRMED or PLAUSIBLE; describe protections in one plain line. Write the report as docs\collab\outbox\collab-review-1.md .

C3. Documentation that matches the code (*files no lane touches*) WHAT_JALEN_CAN_DO.md is stale (it counts 145 tools; main has 165). README.md , SETUP.md , HANDOFF.md , ARCHITECTURE.md and scripts/ help text should state the current commands, the current test count (5,226 collected), the one-machine constraints and the sign-in steps (the Claude brain token, Google, Telegram, the extension). Do not touch CLAUDE.md or ABILITIES.md (generated by scripts/abilities.py). A reviewer should be able to follow SETUP.md on a fresh Windows laptop.

C4. Google and Gmail/Calendar quality (*gmail.py , gcalendar.py , bulkmail.py : no lane touches them*) He runs on the "applicant" Google account; the OAuth consent screen is in Testing mode (7-day token expiry until he publishes it). Audit the Google tools against real requests in data/audit.json1 (read-only copy): which email and calendar requests failed or were answered badly; fix the top causes with fake-API tests; make every error a spoken sentence that says what to do (including the "your Google login expired, run connect_google.py" case, with the 7-day Testing-mode explanation in plain words). Tests never touch the network.

C5. Vault, one-time codes and autofill — a second independent look (*vault.py , otp.py , profile.py , autofill.py*) Recently rewritten: secrets are bound to sites (data\secret_sites.json), a one-time approval is a spoken yes naming the site, a code is typed only into that service's own sign-in host, and fill_login_field is one browser job. Review for bypasses and over-blocking (a legitimate login on a subdomain, a site that redirects to an identity provider), add the tests that are missing, and fix what is wrong. **Never** read, print or store a real secret; use tmp_path vaults.

Order: **C1 first** (it unblocks the owner's real-account check), then C2 (a review is cheap and finds things), then C3-C5 in any order. If you finish your lane, write **one more review** of whatever Wave 4 merged since your last one.

What is explicitly NOT in your lane

Everything in Wave 4's table in 8.4, Telegram sending/stickers/DM tools, folder moves, listening/echo gates, anything in app.py , safety.py , router.py or agent.py , the injection-guard ordering, the Claude hand-off adapter. If you find a bug there, report it in your outbox with a failing test; do not fix it.

10. OWNER DECISIONS LOG (all binding; do not re-ask)

Date	Question	Answer
2026-09-20	How long should Jalen wait for an answer after asking a question?	30 seconds
2026-09-20	Google / extension sign-in	"Sign in once, keep both"
2026-09-30	Wikipedia needs a contact when it refuses automated readers. Which?	His public channel: https://t.me/MLcommunityy (config research.contact)
2026-10-01	Premium emoji in post text	Automatic
2026-10-01	Sticker messages	Only when he asks ("post it with a sticker")
2026-10-01	Voice notes in his DMs: transcribe?	Only when he asks for a specific one (audio goes to Groq)

Date	Question	Answer
2026-10-01	Folders containing <code>.env</code> files (Cafe, SAT TOP) moved C:→D:	Allowed after a NAMED yes: the confirmation lists the protected files carried unread
2026-10-01	Channel posts (pre-approved, no confirmation)	Read the first line and any links aloud first , then post unless he says stop within a few seconds
2026-10-01	Hand tasks to Claude as well as ChatGPT	Yes (he expected to sign in; it appears to have signed itself in — verify)
2026-10-01	Which browser for web tasks	His everyday Chrome , via the extension (Lane B1)
2026-10-01	Research when a site blocks the background fetch	Fast fetch, then his browser, and say so (Lane B2)
2026-10-01	Jalen sends voice messages on his behalf in DMs	Yes (built by the DM branch: Jalen's synthetic TTS voice, RED tier, he confirms the person and the words)
standing	Anything a software engineer could do	Allowed — except deleting online software, accounts or data

11. WHAT HAS BEEN DONE (so you do not redo it)

All of this is merged to `main` (`git log --since=2026-09-30`). Subjects are discoveries, not changes:

Brain and answering - 7594a79 The brain had been signed out for ten days and Jalen read the error aloud as the answer; `jalen check` said "All good" → typed errors (`BrainUnavailable`), once-per-outage notice, retry latch, real `check / readiness` auth probes. - db1c848 His answer to a question switched the injection guard off mid-task → taint survives answers. - 05fc515 "Ed" and "a" were pre-approved destinations (substring match) → exact names. - b79f71b One "yes" approved whichever action waited longest, and Jalen's own "Confirm?" approved itself → `ConfirmAnswer`, bound confirmations, echo defence. - e79cf9f The emergency stop worked only when said the way the config spells it → `is_kill_phrase`, whole-utterance match, both gates.

Safety - cd14483 `never-touch` sees through `~, %VAR%, ..`, long-path prefix. 1693452 ...and through short names, junctions, `::$DATA`, trailing dots, loopback network names. - c8ca2a3 protected domains by hostname. 00f900c GREEN tools that act refused after a read. 1bf36df `web_sign_in` tier + a test that every tool has one. feb3544 per-test taint reset. e97771d router commands in his own words work after a read. ef54882 `web_read` after a read only fetches addresses it was shown. 8973fe5 `run_own_tests` confined to `tests/`; HuggingFace tokenizer files and prose mentioning a password manager no longer refused. 6de6bc3 content search no longer reads protected filenames. 229c9d1 credentials bound to HIS yes and to the site they belong to. 6e98189 a stray "don't" no longer vetoes "stop the coding job".

Browser and research - 9046c62 Jalen's Chrome recovers from a closed tab/window, abandoned jobs, same-tab rules, `browse_to`, `read_browser_page`. 506def6 ...and from a **killed** Chrome (the dead page looked alive). a371945 form steps stay on the inspected site. 53fabf3 **tests no longer open visible Chrome windows** ("the browser keeps failing"). 00fe7b1 / 2f79b67 / 914b22d `web_read`: bounded, PDFs (arXiv papers readable), Wikipedia readable with a contact.

Telegram and jobs - 4815dca HTML posts parsed per call, nothing lost, spoilers never sent plain. 7a18cd9 timeouts say "Not confirmed". 6d34421 coding jobs: brief on stdin/files not `cmd.exe`, honest cancel, locked store.

Diagnostics and hygiene - 2371da0 `jalen check` now checks Google and Telegram. b544b35 the suite stopped writing the real router-miss log. b83d252 unread config keys labelled. 9a94416 CLAUDE.md corrected.

The first wave — built, independently reviewed, merged to main (commits 9f2736b, a955e8e, ce8d92e, e34e790)

1. *posts* — `find_premium_emoji` (exact matches only; a strict single-emoji shape so nothing hidden can ride along), `list_sticker_packs`, `send_sticker` (RED, same destination rules as a text send, and refused after a read even for Saved Messages), read-back of the premium emoji on a sent post. Stickers are sent **only when he asks**.

2. *dms* — catch-up grouped by person, "who needs a reply" ranking that ignores thanks/ok, forward source, voice notes named, `transcribe_voice_note` (AMBER, on request only, audio goes to Groq), `send_voice_message` (RED; the confirmation names the person AND the exact words; Jalen's synthetic voice, not a clone), scoped search, `reply_to`.

3. *files* — `plan_folder_move` / `move_folder` (dry run, copy, verify, switch, delete; resumable; leaves a junction; protected files carried unread after a named yes), launch fixes. **Known data-safety gaps, being fixed in Wave 4: the copy drops Mark-of-the-Web, the hidden attribute and creation dates before the original is deleted; the dry-run reply is a statement; the named yes names only 8 of up to 20 protected files. Treat `move_folder` as not ready for folders whose files matter.**

4. *listening* — measured the real log (of 78 sentences meant for him, 19 were refused by the first count, 40 by the later corrected one), added doors (misheard name, polite request, bare-wake window), a 12 s echo tail. At merge, speech admitted by a door keeps the taint check (`process(text, from_him, door)`). **Open: the answer path's echo window is still 3 s from the end of speech.**

Each of these was checked by an independent reviewer who reproduced the earlier findings; every one returned MERGE_WITH_FIXES, and the fixes that were not already made are Wave 4's first items.

Live QA of the real Jalen in typed mode (55 requests, 2026-10-01): 38 OK, 14 WRONG, 3 REFUSED-WRONGLY. The failures are the source of Lane B items B2-B8: research phrasing → file search; disk questions never answered; "what's using my C drive" (0.7 of 139 GB explained); programs vs windows; "search the web" opens Google and speaks the URL; DDG 202; weather sites blocked; uptime/time-zone/foreground-app missing; `close the calculator`; false injection alarm on arXiv; fused/repeated sentences; replies that leak tool names/HTTP codes.

12. OTHER KNOWN GAPS (not assigned yet; propose before starting)

- A move that outlives a Jalen restart announces nothing; a failed move is announced as "finished: ...".
 - Nothing yet ran against his **real** Telegram for premium emoji / stickers / voice: the first lookup → send → read-back must be done against his own *Saved Messages* and timed, to replace the `[NOT MEASURED]` constants in `stickers.py`.
 - `WHAT_JALEN_CAN_DO.md` is stale (145 tools; there are 155+).
 - Same-origin check for `fill_login_field` on the *second* click; Playwright's auto-wait can retarget after the last URL check (milliseconds).
 - A request that goes to the **brain** after a read ("now open the link"), said without his name, is still refused by the taint check: the refusal says how to get it done. This is the conservative direction on purpose; do not loosen without a poll.
 - Residual: an earlier read in the model's context can still influence a later *clean-turn* post to his channel (the owner chose "read the first line aloud" as the mitigation — Lane A).
-

13. DEFINITION OF DONE and the report you hand back

Done means **all** of these are true and you can show them: 1. A failing test existed first (you saw it fail) and now passes. 2. Focused tests pass; the **full gate** has exactly the known failures (13 in a worktree, 1 in main) and **no change under `data\`**. 3. The change is one commit with a discovery-style subject and a body that says what you did **not** do. 4. You checked it against reality where it touches reality (a live read-only run, or an honest "not

run, because ..."). 5. No secret, no `.env` content, no personal message text in any file or report. 6. Your outbox lists "needs from main" and "not done".

Report template (keep it under one screen):

```
BRANCH: collab/<topic>   COMMIT: <full sha>   TESTS: <total>, <N> new (focused: X pass; full gate:
Y pass, failures = <list>)
DONE (plain language, 3-6 bullets):
NOT DONE / UNVERIFIED (be specific):
NEEDS FROM MAIN (files + exact lines):
RISKS / THINGS I WOULD CHECK NEXT:
POLL FOR THE OWNER (only if truly his decision): <question> | options with one-line consequences
```

14. TRAPS THAT HAVE ALREADY COST HOURS

- **Heredocs mangle regexes and backslashes.** Use Edit/Write. If you must pass a multi-line message to a shell command, use a *quoted* heredoc (`<<'EOF'`) and substitute placeholders with `sed`; an unquoted heredoc executed a backtick inside a commit message today.
- **sleep as a standalone or leading command is blocked** in some Claude Code sessions; wait on a condition (an `until` loop on an output file) or run the long command in the background and read its output file.
- **The harness rewrites paths:** use absolute paths; a relative `.venv/...` fails when passed to `subprocess` on Windows with forward slashes — build it with `os.path.join / abspath`.
- **Windows file locking:** `os.replace` fails with `PermissionError` while any reader has the file open; save code must not swallow that silently.
- **Shared test pollution:** `jarvis/taint.py` is process-wide; `conftest` resets it per test. A new global needs the same treatment or tests will pass or fail depending on file order.
- **A test that supplies the input a bug would have withheld cannot find that bug.** The injection guard was unreachable code for months because a test set the flag by hand. Drive the real path (taint → hook → classify).
- **Do not trust a reviewer or a subagent summary blindly**, including this document: if it names a file, function or number, **VERIFY** it before acting.
- **pytest's tmp_path is named after the test;** a test called `..._secret_...` puts the word "secret" in the folder name and trips the `*secret*` protected-name pattern.
- **Windows chrome.exe exits** when the last window closes; Jalen's own Chrome must be re-launched, and killing it leaves a "live-looking" dead page (fixed, but remember the shape).
- **8.3 short names, junctions and Documents and Settings are real on this machine;** paths are compared after `os.path.realpath`.
- **Voice cannot be tested on the owner's microphone from here.** Anything about listening, echo, barge-in or the orb is verified by corpus tests over the audit log plus the owner's own use.

15. FIRST 10 MINUTES CHECKLIST

1. `git -C C:\Users\user\Desktop\Jarvis-setup\jarvis log -5 --format='%h %s'` — confirm main is at (or after) `e34e790`; read `docs\collab\STATUS.md` if it exists.
2. `.\jalen.ps1 check` — expect: Claude signed in, Google (may say "refusing" after 2026-10-08), Telegram signed in, "All good".

3. Create your worktree (8.2). Create `docs\collab\outbox\collab-live-check.md` on your branch with a one-line plan.
4. Start C1: read `jarvis/tools/messaging.py`, `jarvis/tools/stickers.py`, `tests/_telegram_fakes.py`, `tests/test_sticker_tools.py`, `tests/test_telegram_voice.py`; run those tests alone; write down which functions the script will call and what each needs from the Telegram client.
5. Then work the loop in 8.6 on Lane C, starting with C1. Report at the first meaningful commit, not at the end.

Tone of every message you send the owner: short, warm, concrete, honest about what is and is not verified. Never pretend something works that you have not run.